

Can AI Agents Really Complete RTL-to-GDS? Lessons from Benchmarking Tool-Interactive EDA Workflows

Jinyuan Deng
Zhejiang University
Hangzhou, Zhejiang, China

Zhengrui Chen
Zhejiang University
Hangzhou, Zhejiang, China

Xufeng Wei
Zhejiang University
Hangzhou, Zhejiang, China

Tianyu Xing
Zhejiang University
Hangzhou, Zhejiang, China

Chenyi Wen
Zhejiang University
Hangzhou, Zhejiang, China

Cheng Zhuo
Zhejiang University
Hangzhou, Zhejiang, China
czhuo@zju.edu.cn

Abstract

LLM-driven agent systems have emerged as a promising paradigm for electronic design automation (EDA), demonstrating strong potential for automating complex design workflows. However, existing evaluations primarily examine individual language models on isolated EDA tasks, providing limited insight into how different agent systems perform across complete EDA flows. In this work, we present FluxBench, a systematic evaluation of AI agents on end-to-end EDA workflows under unified prompts, tool environments, and technology library settings. Our evaluation covers representative scenarios, including RTL generation with open-source toolchains and an RTL-to-GDS flow using closed-source commercial EDA tools for industrial applications. Through these workflows, we assess agents' capabilities in RTL code generation, iterative repair, tool-feedback utilization, logic synthesis, placement and routing (P&R), and Engineering Change Order (ECO) automation. To further characterize the efficiency of agent systems, we introduce Token ROI, a cost-efficiency metric that measures effective improvements in EDA artifacts relative to token usage and runtime cost. Experimental results show that, even when built on the same foundation model, different agent system architectures can exhibit performance gaps of up to 86.27%. Moreover, among systems with comparable task performance, Token ROI can differ by as much as 105.92%. In the RTL-to-GDS flow using PicoRV32 as a case study, FluxEDA achieves an end-to-end score of up to 97.94, outperforming Claude Code equipped with domain-specific EDA skills by up to 8.39%. These results indicate that domain-specific skills alone are insufficient to improve agent performance in large-scale EDA scenarios. Instead, both agent system design and foundation model capability play critical roles in enabling effective automated EDA workflows.

Keywords

AI Agents, EDA Workflows, Benchmark

1 Introduction

Large language models (LLMs) have recently shown strong potential in electronic design automation (EDA). Prior work has explored LLMs for Verilog generation, RTL assistance, EDA scripting, bug summarization, and domain-specific chip-design support [6, 12, 16, 18]. Building on these advances, recent studies have further moved toward LLM-based agents that can interact with EDA tools, decompose design tasks, execute scripts, analyze feedback,

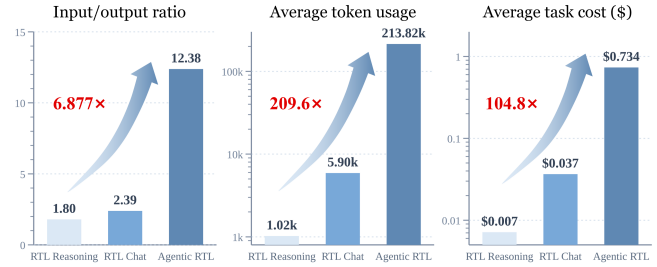


Figure 1: Comparison of token usage across three RTL-task settings: *RTL Reasoning*, *RTL Chat*, and *Agentic RTL*. *RTL Reasoning* refers to single-turn problem solving for RTL generation without tool interaction, *RTL Chat* denotes multi-turn dialogue for RTL-related problem solving, and *Agentic RTL* represents agentic RTL design with tool use.

and iteratively refine designs [8, 11, 13, 14]. This shift suggests a broader transition from LLMs as passive design assistants to autonomous agents capable of participating in end-to-end EDA flows.

Unlike general natural-language or coding tasks, EDA tasks are inherently tool-centric, requiring agents to invoke simulators, synthesizers, physical-design tools, and log analyzers through repeated multi-round interactions. Such tool use not only increases task complexity but also introduces token and cost overhead. As shown in Figure 1, moving from single-turn RTL reasoning to multi-turn RTL chat and agentic RTL design increases the input/output ratio from 1.80 to 12.38, average token usage from 1.02K to 213.82K, and average task cost from \$0.007 to \$0.734. To better advance AI agents for EDA, high-quality benchmarks are therefore essential. Existing benchmarks, such as VerilogEval and RTLLM, have provided valuable evaluation platforms for LLM-based Verilog/RTL generation, mainly focusing on functional correctness, syntax, and design quality [7, 9]. However, they are primarily designed for model-level generation tasks and provide limited support for reliably evaluating token consumption and multi-round tool-use behaviors. Consequently, the field still lacks a standardized, cost-aware, and tool-interactive benchmark for process-level evaluation of EDA agents in realistic end-to-end optimization flows.

To fill this gap, we present FluxBench, which, to the best of our knowledge, provides the first systematic benchmarking of AI agents in realistic, end-to-end, and long-horizon EDA scenarios. FluxBench not only includes diverse conventional LLM-for-EDA

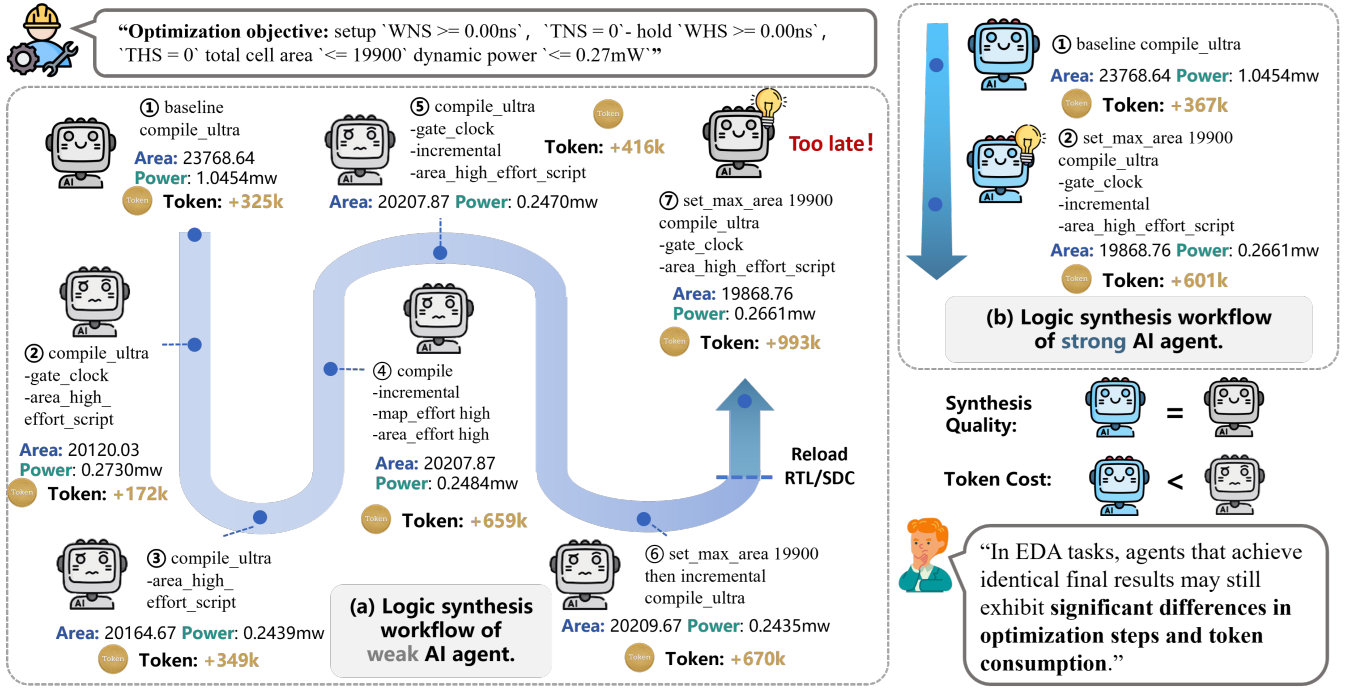


Figure 2: Logic synthesis workflows of AI agents with different capabilities. A weak agent may require trial-and-error optimization and high token consumption, while a strong agent can reach comparable QoR with fewer steps and lower token cost.

benchmarks and representative classical RTL generation tasks based on open-source tools, but also further introduces closed-source-tool-based PicoRV32 RTL-to-GDS tasks to better reflect practical and complex EDA workflows. In addition to conventional agent-level metrics and EDA-specific task metrics, FluxBench introduces Token ROI, a cost-efficiency metric for evaluating the return on agent usage. Specifically, we first define the task reward for each EDA task according to its design objective, and then divide it by the agent’s time and token consumption. Token ROI reflects how much useful EDA optimization improvement an agent can achieve per unit token and time cost, which is particularly important for long-horizon EDA tasks where tool interaction and iterative debugging can lead to substantial token and time overhead.

Our benchmarking results show that AI agents producing similar final design outcomes can still exhibit substantially different optimization trajectories. In particular, the final Token ROI varies by up to 105.92× across agents. As shown in Figure 2, stronger agents typically reach the optimization target within only a few iterations, whereas weaker agents often require many more trials to identify a valid or near-optimal configuration. This leads to significant differences in both token consumption and runtime cost, highlighting the importance of process-level evaluation beyond final design quality. We further evaluate the effectiveness of domain-specific EDA skills. The results show that such skills provide moderate improvements under loose timing constraints, but offer limited benefits under more stringent timing requirements. In contrast, under the same foundation model, FluxEDA achieves an average performance improvement of 57.00 points. These results suggest that both the agent

architecture and the underlying foundation model are key factors for building effective automated EDA workflows.

Our contributions can be summarized as follows:

- To the best of our knowledge, FluxBench provides the first systematic benchmarking of AI agents in realistic, end-to-end, and long-horizon EDA flows.
- We benchmark AI agents across conventional LLM-for-EDA evaluation tasks, classical RTL generation tasks using open-source tools, and PicoRV32 RTL-to-GDS flows using closed-source commercial tools, enabling comprehensive evaluation across EDA workflows of different scales and complexities.
- We propose Token ROI, a cost-efficiency metric that measures the optimization gain toward EDA task objectives per unit token and time cost.

2 Related Work

LLMs and Benchmarks in EDA. LLMs show significant potential in streamlining EDA processes [16, 18], with models like ChipNeMo [6] and VeriGen [12] focusing on domain adaptation and direct code generation. To evaluate these capabilities, early benchmarks such as RTLLM [9] were introduced for basic RTL generation tasks. Recently, the scope of evaluation has expanded significantly: RealBench [5] assesses models on real-world IP-level designs, and CVDP [10] introduces comprehensive Verilog design problems to test broader reasoning skills. However, despite this growing diversity, these datasets primarily evaluate static outcomes or single-turn

generations. A critical gap remains: existing benchmarks still provide limited support for evaluating autonomous *agents* in realistic end-to-end, long-horizon, and tool-interactive EDA flows.

Autonomous Agents for EDA. To overcome single-turn limitations, recent research has shifted toward autonomous agents capable of managing multi-step EDA workflows. Frameworks like ChatEDA [13] and AutoEDA [8] introduced LLM-driven task planning and modular tool automation. Building on this paradigm, FluxEDA [3] has emerged as a prominent framework, offering robust, agent-driven workflow automation to effectively streamline the hardware design process.

3 Benchmark Construction

3.1 Execution Environment

As illustrated in Figure 3, we encapsulate all tasks in a containerized environment to reduce the impact of host-dependent variation on evaluation results. The container provides a complete open-source EDA toolchain for RTL generation tasks. For tasks requiring commercial EDA tools or restricted PDKs, the benchmark does not directly distribute these dependencies; instead, they are accessed from the user environment through read-only mounts at runtime, balancing reproducibility and compliance. Each task runs in an independent agent session, where the agent can access only the task description, public inputs, and editable files, without visibility into hidden tests, reference solutions, or internal scoring data.

3.2 Data Collection

The selected tasks mainly consist of RTL generation and RTL-to-GDS tasks. The RTL generation tasks are selected from the spec-to-RTL subsets of RealBench [5] and CVDV [10]. RealBench contains three circuit families: AES, e203_hbirdv2, and SDC. We classify the e203_hbirdv2 tasks as hard and the tasks from the other two families as easy. For CVDV, we select 112 tasks based on their scores, including 18 hard tasks and 94 easy tasks. These tasks require agents to generate RTL modules from natural-language specifications and validate their functionality using testbenches, thereby evaluating their hardware semantic understanding, Verilog/SystemVerilog programming, and simulation debugging capabilities.

For RTL-to-GDS tasks, we select the PicoRV32 design with a 55 nm technology node. PicoRV32 is a compact open-source RISC-V processor core whose datapath and control logic are more representative than those of small toy circuits, while its moderate scale keeps full physical implementation and iterative optimization tractable. The mature 55 nm process exposes agents to realistic synthesis, timing, floorplanning, placement, routing, and ECO constraints without excessive complexity. This combination enables a controlled and meaningful evaluation of end-to-end implementation capabilities.

3.3 Quality Control and Reproducibility

The agent can only access public materials at runtime, while reference results and final evaluation logic are used by the evaluator only after the task is completed. Each task runs independently from a consistent initial workspace, and all commands, logs, reports, and final artifacts are preserved. For commercial-tool tasks, missing

tools, files, or process data are recorded as environment failures rather than being masked by substitute flows.

4 Experiments

4.1 Experimental Setup

Agent architectures. We evaluate three representative agentic coding frameworks: FluxEDA-Agent, Claude Code, and CC+EDA Skills. FluxEDA-Agent targets EDA-oriented agent workflows [3], while Claude Code serves as a general-purpose coding agent [1]. CC+EDA Skills extends Claude Code with domain-specific skills, which provide domain-specific IC/EDA workflow knowledge [4].

Models. We use three representative language models as the underlying reasoning and coding backbones: Deep Seek-V4 Flash, GLM-5.2, and Claude Sonnet 5. These models represent some of the most widely used and advanced systems in current agentic coding workflows, and are selected for their strong capabilities in coding, reasoning, long-context understanding, and tool use [2, 15, 17].

Task inputs and constraints. For RTL generation tasks, agents are provided with natural-language functional specifications, together with a small set of auxiliary scripts and RTL code. For RTL-to-GDS tasks, the technology files are provided locally, and two timing constraints are considered: a loose constraint of 350 MHz and a tight constraint of 700 MHz, both specified through SDC files. The die size is fixed in the prompt, requiring agents to optimize design quality under the given timing constraint.

4.2 Metric Definition: Token Return on Investment in Agentic EDA

We define *Token Return on Investment* (Token ROI) as a task-normalized measure of EDA return per unit token investment and per unit execution time. It evaluates how efficiently an agent converts token-based cost and execution time into task-specific EDA outcomes.

Token investment. For task k , let the agent trajectory be

$$\tau_k = (p_k, \{(y_t, o_t)\}_{t=1}^T),$$

where p_k is the initial task prompt, y_t is the model response, and o_t is the tool observation at step t . The token investment is defined as the total token-based cost over the full execution:

$$C_{\text{token}}(\tau_k) = \sum_{t=1}^T \sum_{j \in \mathcal{J}} n_t^{(j)} \cdot r^{(j)}, \quad (1)$$

where $\mathcal{J}$ denotes the set of token types, such as input, output, cached, reasoning, and tool-observation tokens. Here, $n_t^{(j)}$ is the number of type- j tokens consumed at step t , and $r^{(j)}$ is the corresponding unit price.

Time investment. In addition to token investment, we also consider execution time as an important resource investment in agentic EDA. For task k , the time investment is defined as the total wall-clock time required for the agent to complete the trajectory τ_k :

$$C_{\text{time}}(\tau_k) = t_k^{\text{end}} - t_k^{\text{start}}, \quad (2)$$

where t_k^{start} and t_k^{end} denote the start and end times of the task execution, respectively. This time cost captures the overall execution efficiency of the agent during interactive reasoning, tool use, result parsing, and iterative refinement.

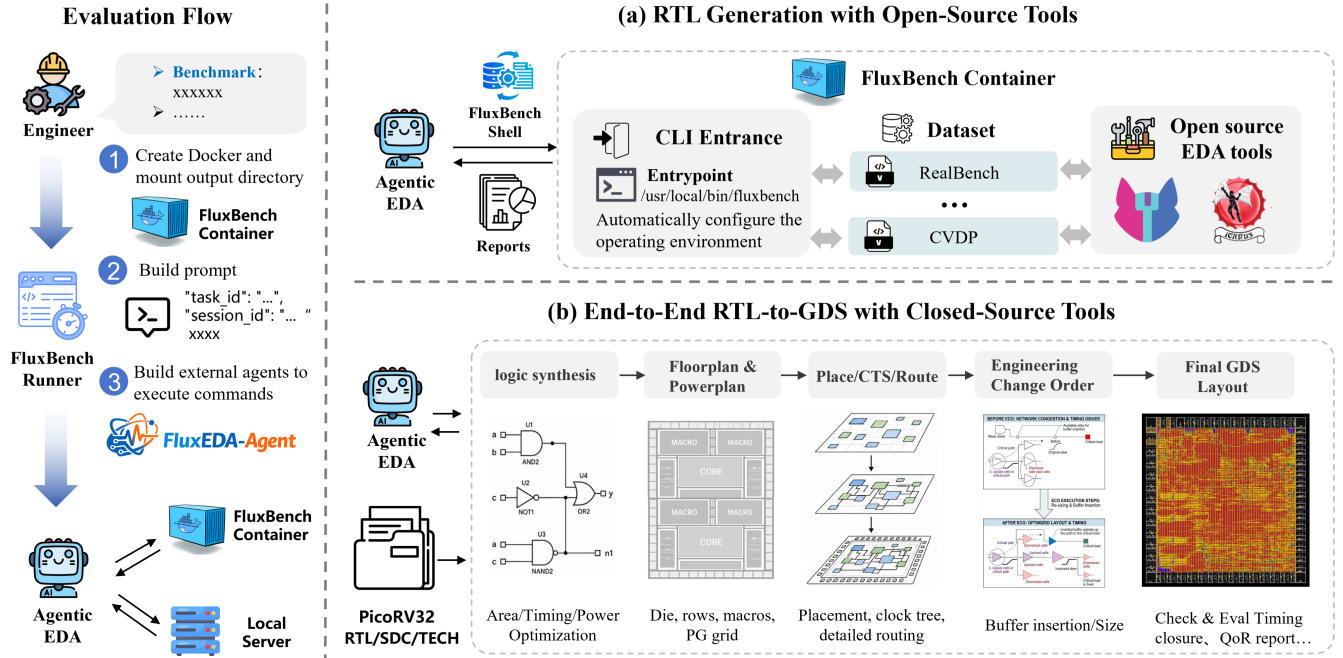


Figure 3: Benchmark Construction and Evaluation Flow of FluxBench. The workflow begins with benchmark setup, containerized environment preparation for open-source tool tasks, and prompt generation, followed by agentic EDA execution through FluxEDA-Agent. FluxBench supports RTL generation with open-source tools and end-to-end RTL-to-GDS evaluation with closed-source tools, covering synthesis, floorplanning, placement/routing, ECO, and final GDS/report generation.

Table 1: Task-specific EDA return definitions for Token ROI evaluation.

Task Type	Stage	Results / Variables	Score Definition
RTL Generation	RTL generation	p_{pass} : functional test pass rate; p_{compile} : compile pass rate; p_{synth} : synthesis pass rate.	$R_{\text{RTL}} = 80 p_{\text{pass}} + 10 p_{\text{compile}} + 10 p_{\text{synth}}$
	Logic synthesis	Binary completion indicators $I_i \in \{0, 1\}$; I_{cg} : clock gating; I_{netlist} : synthesized gate-level netlist generation.	$R_{\text{syn}} = 50 I_{\text{cg}} + 50 I_{\text{netlist}}$
	Place-and-route	Binary completion indicators $I_i \in \{0, 1\}$: I_{IO} : I/O placement; I_{std} : standard-cell placement; I_{CTS} : clock-tree synthesis; I_{route} : routing.	$R_{\text{PR}} = 25 (I_{\text{IO}} + I_{\text{std}} + I_{\text{CTS}} + I_{\text{route}})$
RTL-to-GDS	Engineering change order	$I_{\text{PAECO}} \in \{0, 1\}$: binary completion indicator for physical-aware ECO.	$R_{\text{ECO}} = 100 I_{\text{PAECO}}$
	Final result	P : final power; I_{PAECO} : physical-aware ECO indicator; w_s^+, t_s^+ : post-ECO setup WNS and TNS; w_h^+, t_h^+ : post-ECO hold WNS and TNS. Each $S_i(\cdot)$ is a task-specific scoring function normalized to $[0, 100]$.	$R_{\text{final}} = \lambda_P S_P(P) + \lambda_{\text{aware}} S_{\text{aware}}(I_{\text{PAECO}})$ $+ \lambda_{\text{sWNS}} S_{\text{sWNS}}(w_s^+) + \lambda_{\text{sTNS}} S_{\text{sTNS}}(t_s^+)$ $+ \lambda_{\text{hWNS}} S_{\text{hWNS}}(w_h^+) + \lambda_{\text{hTNS}} S_{\text{hTNS}}(t_h^+)$ $\lambda_P + \lambda_{\text{aware}} + \lambda_{\text{sWNS}} + \lambda_{\text{sTNS}} + \lambda_{\text{hWNS}} + \lambda_{\text{hTNS}} = 1$

EDA return. For each task k , we use the normalized task-specific score $S_k \in [0, 1]$ as the EDA return. The detailed scoring rules are given in Table 1 and Appendix A.

Token ROI. Token ROI is defined as the EDA return per unit token investment and per unit execution time:

$$\text{TokenROI}_k(\tau_k) = \frac{S_k}{C_{\text{token}}(\tau_k) \cdot C_{\text{time}}(\tau_k)}. \quad (3)$$

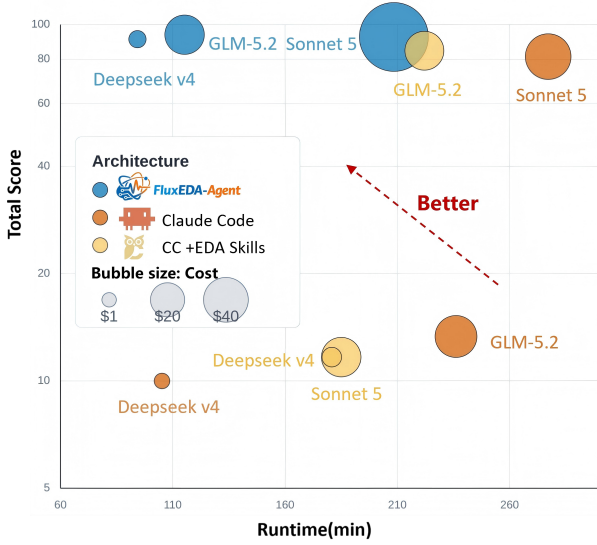


Figure 4: Performance-runtime-cost comparison across agent architectures: higher total scores and shorter runtimes are better, while bubble size indicates cost.

Normalized Token ROI. To compare resource efficiency across tasks, we normalize token investment and execution time using task-specific reference bounds. For token investment, the actual token-based cost is first clipped:

$$\tilde{C}_{\text{token},k}(\tau_k) = \min \left(\max \left(C_{\text{token}}(\tau_k), C_k^{\text{best}} \right), C_k^{\text{worst}} \right). \quad (4)$$

It is then linearly mapped to $[1, 100]$:

$$\hat{C}_{\text{token},k}(\tau_k) = 1 + 99 \frac{\tilde{C}_{\text{token},k}(\tau_k) - C_k^{\text{best}}}{C_k^{\text{worst}} - C_k^{\text{best}}}. \quad (5)$$

Similarly, execution time $C_{\text{time}}(\tau_k)$ is normalized using task-specific reference bounds T_k^{best} and T_k^{worst} with the same clipped linear mapping scheme, but mapped to the range $[1, 5]$, yielding $\hat{C}_{\text{time},k}(\tau_k)$.

Finally, the normalized Token ROI is defined as

$$\text{NormTokenROI}_k(\tau_k) = \frac{S_k}{\hat{C}_{\text{token},k}(\tau_k) \cdot \hat{C}_{\text{time},k}(\tau_k)}. \quad (6)$$

4.3 Main Results

RTL generation. Table 2 reports the results of three representative agent architectures combined with different foundation models on the easy and hard subsets of RealBench and CVDp. Overall, the results reveal substantial differences across benchmark difficulty levels. CVDp-Easy is nearly saturated, with six of the nine evaluated configurations achieving a perfect score of 100.00 and the remaining scores exceeding 98.00. In contrast, RealBench remains considerably more challenging: the best scores on its easy and hard subsets are only 57.10 and 53.58, respectively, both achieved by FluxEDA with GLM-5.2. CVDp-Hard also retains strong discriminative capability, where scores range from 52.35 to 100.00, and FluxEDA with GLM-5.2 is the only configuration that achieves a perfect score.

From the perspective of efficiency, the architectures and foundation models exhibit clear trade-offs in score, ROI, cost, and runtime. DeepSeek-V4 consistently provides the most favorable cost profile, with costs generally below 18 RMB and substantially higher ROI than GLM-5.2 and Claude Sonnet 5. When paired with DeepSeek-V4, FluxEDA improves the RealBench-Easy, RealBench-Hard, and CVDp-Hard scores from 41.26, 22.00, and 69.58 under vanilla Claude Code to 46.62, 32.74, and 78.04, respectively. It also reduces the runtime on all four subsets, while lowering the cost on RealBench-Hard and CVDp-Hard from 9.83 and 7.76 RMB to 9.50 and 5.68 RMB. CC+EDA Skills shows more mixed behavior: it improves DeepSeek-V4 on RealBench-Hard from 22.00 to 31.09 while reducing the cost from 9.83 to 8.77 RMB, but does not consistently improve performance on CVDp. FluxEDA with GLM-5.2 achieves the strongest overall scores, although this advantage is accompanied by substantially higher costs and longer runtimes.

RTL-to-GDS. Table 3 summarizes the end-to-end PicoRV32 RTL-to-GDS results under the loose 350 MHz and tight 700 MHz timing targets. Overall, FluxEDA delivers the strongest and most consistent optimization performance across both settings. Under the loose target, FluxEDA with DeepSeek-V4 achieves the highest score of 97.94 and the highest ROI of 39.91. Although Claude Code with DeepSeek-V4 attains the lowest cost and shortest runtime in this setting, its score is only 13.33, indicating a substantial trade-off in final design quality. Under the tight 700 MHz target, FluxEDA with GLM-5.2 obtains the best score of 93.70, while FluxEDA with DeepSeek-V4 achieves both the highest ROI of 48.06 and the shortest runtime of 94.28. Notably, all three models paired with FluxEDA achieve scores above 90 under both timing targets, demonstrating strong robustness across different LLM backends. In contrast, the Claude Code and CC+EDA Skills baselines exhibit considerably greater performance variation across models and timing constraints. These results demonstrate that FluxEDA provides a favorable balance between final QoR and optimization efficiency, particularly in challenging end-to-end physical design scenarios. Meanwhile, Figure 4 further shows that FluxEDA achieves these improvements with a reasonable token overhead.

5 Observation

5.1 The Role of Skills in Agentic EDA

Observation 1: The impact of skills is limited. Although the domain-specific EDA skills improves performance over Claude Code on easy RTL generation tasks and the 350 MHz RTL-to-GDS task, it shows almost no advantage under the more stringent 700 MHz constraint and may even have a negative effect. This suggests that, in realistic EDA RTL-to-GDS scenarios, simply relying on skills does not necessarily lead to better outcomes. Instead, stronger foundation models and more effective system architectures are the key to achieving stable capability improvements.

5.2 Error Patterns in the PR Stage

Observation 2: Innovus Tcl Command compatibility. In the PR stage, Innovus Tcl Command compatibility is the most prominent error category, accounting for 31.7% of all PR error events, as shown in Figure 5. This indicates that agentic PR is not only constrained

Table 2: Results on open-source RTL generation tasks. ROI denotes the average normalized Token ROI. Cost reports the total RMB cost of completing all tasks in each benchmark subset. Runtime denotes the total running time in minutes required to complete all tasks in the benchmark subset. CC+EDA Skills denotes Claude Code equipped with domain-specific EDA skills.

Architecture	Model	RTL Generation															
		RealBench [5]								CVDP [10]							
		Easy				Hard				Easy				Hard			
		Score	ROI	Cost (¥)	Runtime (min)	Score	ROI	Cost (¥)	Runtime (min)	Score	ROI	Cost (¥)	Runtime (min)	Score	ROI	Cost (¥)	Runtime (min)
Claude Code	DeepSeek-V4	41.26	22.66	5.17	297.98	22.00	11.38	9.83	512.71	100.00	73.93	12.08	825.89	69.58	33.10	7.76	420.00
	GLM-5.2	50.91	0.55	108.58	485.65	47.06	0.36	287.39	1066.45	100.00	2.64	302.38	1488.29	94.71	0.45	143.61	621.75
	Claude Sonnet 5	48.72	0.89	188.60	231.75	50.37	0.55	389.56	520.83	100.00	2.34	402.49	501.94	90.00	0.77	164.78	233.38
CC+EDA Skills	DeepSeek-V4	41.11	23.10	6.02	330.38	31.09	18.22	8.77	474.78	98.04	74.28	13.08	783.91	52.35	26.04	6.22	404.50
	GLM-5.2	52.61	1.15	94.83	454.44	51.52	0.53	257.87	1007.69	99.04	2.49	275.27	1462.43	84.44	0.37	103.83	544.99
	Claude Sonnet 5	54.66	0.77	221.23	261.18	47.96	0.42	395.37	516.32	100.00	2.31	430.76	533.01	80.00	0.63	214.54	315.55
FluxEDA	DeepSeek-V4	46.62	22.63	5.28	225.85	32.74	18.70	9.50	381.25	100.00	70.72	17.05	762.98	78.04	28.51	5.68	244.30
	GLM-5.2	57.10	1.66	148.92	765.97	53.58	3.00	355.83	1576.77	98.09	5.71	447.34	1662.39	100.00	0.54	243.82	653.42
	Claude Sonnet 5	55.47	0.41	302.80	332.35	53.02	0.30	570.55	604.58	100.00	1.82	632.18	619.23	90.00	0.45	363.63	447.08

Table 3: Results on end-to-end RTL-to-GDS flows.

Architecture	Model	Loose (350 MHz)				Tight (700 MHz)			
		Score	ROI	Cost (¥)	Runtime (min)	Score	ROI	Cost (¥)	Runtime (min)
Claude Code	DeepSeek-V4	13.33	12.71	0.73	58.78	10.00	5.83	1.39	105.19
	GLM-5.2	13.33	0.12	224.81	302.30	13.33	0.32	96.89	236.09
	Claude Sonnet 5	87.05	1.26	138.58	284.23	81.33	1.36	122.62	277.18
CC+EDA Skills	DeepSeek-V4	11.67	4.81	4.30	111.70	11.67	2.75	6.35	180.81
	GLM-5.2	95.97	6.22	56.53	129.84	84.47	2.61	78.05	222.04
	Claude Sonnet 5	11.67	0.42	114.58	118.81	11.67	0.40	81.77	185.00
FluxEDA	DeepSeek-V4	97.94	39.91	3.76	120.83	90.99	48.06	3.04	94.28
	GLM-5.2	97.72	5.77	72.06	110.63	93.70	4.84	80.51	115.27
	Claude Sonnet 5	96.41	0.68	325.85	252.65	92.36	0.72	348.36	208.48

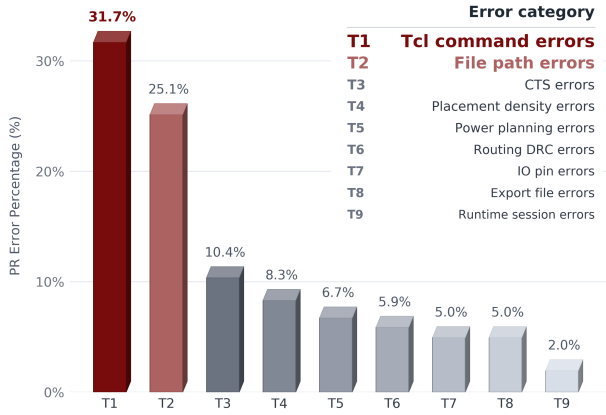


Figure 5: Distribution of PR-stage error categories. T1 corresponds to Innovus Tcl Command compatibility errors, which form the largest category by percentage.

by floorplan search, placement, CTS, or routing, but also by a more basic requirement: the agent must generate Tcl Commands that

Table 4: Percentage breakdown of Innovus Tcl Command compatibility errors by framework and model.

Framework	DeepSeek-V4	GLM-5.2	Claude Sonnet 5
Claude Code	48.7%	28.0%	2.7%
CC+EDA Skills	10.0%	7.7%	2.9%
FluxEDA	0.0%	0.0%	0.0%

match the current Innovus version, execution mode, MMMC init flow, and report and export interfaces. The detailed breakdown in Table 4 shows that this category is concentrated in Claude Code runs, especially Claude Code + DeepSeek-V4 and GLM-5.2, which contribute 48.7% and 28.0% of these errors, respectively. By contrast, the corresponding CC+EDA Skills shares decrease to 10.0% for DeepSeek-V4 and 7.7% for GLM-5.2, and FluxEDA contributes 0.0% across all models. These results suggest that structured workflows and wrapped tool interfaces can substantially reduce direct exposure to low-level Tcl Command and tool-version mismatches. At the same time, the absence of this category in FluxEDA does not imply that the PR flow has no failure modes; rather, failures shift

Table 5: Completion status of different stages in the RTL-to-GDS flow. ✓ indicates completed, and ✗ indicates not completed. Physical-aware ECO denotes ECO modifications that incorporate physical information, such as cell locations.

Architecture	Model	Frequency	Logic Synthesis		Place and Route				ECO
			Clock Gating	Netlist Generation	I/O Placement	Standard-Cell Placement	CTS	Routing	Physical-Aware ECO
Claude Code	DeepSeek-V4	700 MHz	✓	✓	✗	✓	✗	✓	✗
		350 MHz	✓	✓	✓	✓	✓	✓	✗
	GLM-5.2	700 MHz	✓	✓	✓	✓	✓	✓	✗
		350 MHz	✓	✓	✓	✓	✓	✓	✗
	Claude Sonnet 5	700 MHz	✓	✓	✓	✓	✓	✓	✓
		350 MHz	✓	✓	✓	✓	✓	✓	✓
CC+EDA Skills	DeepSeek-V4	700 MHz	✓	✓	✗	✓	✓	✓	✗
		350 MHz	✓	✓	✗	✓	✓	✓	✗
	GLM-5.2	700 MHz	✓	✓	✓	✓	✓	✓	✓
		350 MHz	✓	✓	✓	✓	✓	✓	✓
	Claude Sonnet 5	700 MHz	✓	✓	✗	✓	✓	✓	✗
		350 MHz	✓	✓	✗	✓	✓	✓	✗
FluxEDA	DeepSeek-V4	700 MHz	✓	✓	✓	✓	✓	✓	✓
		350 MHz	✓	✓	✓	✓	✓	✓	✓
	GLM-5.2	700 MHz	✓	✓	✓	✓	✓	✓	✓
		350 MHz	✓	✓	✓	✓	✓	✓	✓
	Claude Sonnet 5	700 MHz	✓	✓	✓	✓	✓	✓	✓
		350 MHz	✓	✓	✓	✓	✓	✓	✓

toward higher-level API session, path configuration, and design-convergence issues. Therefore, for agentic EDA in the PR stage, improving model capability alone is insufficient: constraining and validating Innovus Tcl Command generation is critical for reducing debugging cost and improving workflow stability.

5.3 Stability of the RTL-to-GDS Flow

Observation 3: The MCP-based architecture of FluxEDA provides substantially higher process stability than conventional CLI-based approaches. As shown in Table 5, FluxEDA successfully completes all stages of the RTL-to-GDS flow, including logic synthesis, place-and-route, and physical-aware ECO, across all evaluated models and target frequencies. In contrast, the conventional Claude Code configuration exhibits notable instability at several stages. For example, Claude Code with DeepSeek-V4 fails to complete I/O placement, CTS, and physical-aware ECO under the 700 MHz target, while several other configurations also terminate before completing the ECO stage. Moreover, introducing the domain-specific skills does not fundamentally resolve this issue: under the Claude Code + EDA Skills configuration, DeepSeek-V4 and Claude Sonnet 5 still fail to complete I/O placement and physical-aware ECO at both target frequencies. This indicates that Skills can improve the model’s understanding of EDA procedures and provide useful domain guidance, but they cannot fully compensate for the structural limitations of CLI-based interaction, such as weak state management, unstructured tool feedback, and limited error recovery. Although FluxEDA may consume more tokens because MCP introduces structured tool descriptions, execution feedback, and additional interactions, this overhead is acceptable for EDA workflows. Since RTL-to-GDS flows involve long execution chains,

strong inter-stage dependencies, and low error tolerance, the improved reliability and end-to-end stability provided by MCP are more important than minimizing token consumption.

6 Conclusion

This paper presents FluxBench to systematically evaluate AI agents in realistic end-to-end EDA workflows. It covers RTL generation and industrial PicoRV32 RTL-to-GDS tasks, and introduces Token ROI to jointly measure performance, cost, and runtime efficiency. Results show that foundation models and agent architectures are key factors, while domain-specific Skills alone are insufficient for complex industrial EDA flows, as agents may still fail at critical steps of the RTL-to-GDS process.

References

- [1] Anthropic. 2026. Claude Code Documentation. <https://docs.anthropic.com/en/docs/claude-code/overview>. Accessed: 2026-06-27.
- [2] Anthropic. 2026. Introducing Claude Sonnet 5. <https://www.anthropic.com/news/claude-sonnet-5>. Accessed: 2026-07-09.
- [3] Zhengrui Chen, Zixuan Song, Yu Li, Qi Sun, and Cheng Zhuo. 2026. FluxEDA: A Unified Execution Infrastructure for Stateful Agentic EDA. *arXiv preprint arXiv:2603.25243* (2026).
- [4] ICkylin Team. 2026. IC Agent Hub: IC-Oriented Agent Skills by ICkylin. <https://www.icagenthub.com/skills>. Accessed: 2026-07-09. IC-oriented skill marketplace developed by Zhongke Qixin / ICkylin; homepage: <https://www.ickylin.com/>.
- [5] Pengwei Jin, Di Huang, Chongxiao Li, Shuyao Cheng, Yang Zhao, Xinyao Zheng, Jiaguo Zhu, Shuyi Xing, Bohan Dou, Rui Zhang, et al. 2025. Realbench: Benchmarking verilog generation models with real-world ip designs. *arXiv preprint arXiv:2507.16200* (2025).
- [6] Mingjie Liu, Teodor-Dumitru Ene, Robert Kirby, Chris Cheng, Nathaniel Pinckney, Rongjian Liang, Jonah Alben, Himyanshu Anand, Sanmitra Banerjee, Ismet Bayraktaroglu, et al. 2023. Chipnemo: Domain-adapted llms for chip design. *arXiv preprint arXiv:2311.00176* (2023).
- [7] Mingjie Liu, Nathaniel Pinckney, Bruce Khailany, and Haoxing Ren. 2023. VerilogEval: Evaluating large language models for verilog code generation. In *2023*

- IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 1–8.
- [8] Yiyi Lu, Hoi Ian Au, Junyao Zhang, Jingyu Pan, Guanglei Zhou, Yiting Wang, Jingwei Sun, Ang Li, Jianyi Zhang, Hai Li, et al. 2025. AutoEDA: Enabling EDA flow automation through microservice-based LLM agents. *arXiv preprint arXiv:2508.01012* (2025).
 - [9] Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. 2024. Rtl1m: An open-source benchmark for design rtl generation with large language model. In *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 722–727.
 - [10] Nathaniel Pinckney, Chenhui Deng, Chia-Tung Ho, Yun-Da Tsai, Mingjie Liu, Wenfei Zhou, Brucek Khailany, and Haoxing Ren. 2025. Comprehensive Verilog design problems: A next-generation benchmark dataset for evaluating large language models and agents on rtl design and verification. *arXiv preprint arXiv:2506.14074* (2025).
 - [11] The Verkor Team, Ravi Krishna, Suresh Krishna, and David Chin. 2026. Design Conductor: An agent autonomously builds a 1.5 GHz Linux-capable RISC-V CPU. *arXiv preprint arXiv:2603.08716* (2026).
 - [12] Shailja Thakur, Baleegh Ahmad, Hammond Pearce, Benjamin Tan, Brendan Dolan-Gavitt, Ramesh Karri, and Siddharth Garg. 2024. Verigen: A large language model for verilog code generation. *ACM Transactions on Design Automation of Electronic Systems* 29, 3 (2024), 1–31.
 - [13] Haoyuan Wu, Zhuolun He, Xinyun Zhang, Xufeng Yao, Su Zheng, Haisheng Zheng, and Bei Yu. 2024. Chateda: A large language model powered autonomous agent for eda. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 43, 10 (2024), 3184–3197.
 - [14] Haoyuan Wu, Haisheng Zheng, Zhuolun He, and Bei Yu. 2025. Divergent thoughts toward one goal: Llm-based multi-agent collaboration system for electronic design automation. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 1710–1721.
 - [15] Anyi Xu, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, Chenchen Ling, et al. 2026. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence. *arXiv preprint arXiv:2606.19348* (2026).
 - [16] Kangwei Xu, Denis Schwachhofer, Jason Blocklove, Ilia Polian, Peter Domanski, Dirk Pflüger, Siddharth Garg, Ramesh Karri, Ozgur Sinanoglu, Johann Knechtel, et al. 2025. Large Language Models (LLMs) for Electronic Design Automation (EDA). *arXiv preprint arXiv:2508.20030* (2025).
 - [17] Z.AI. 2026. GLM-5.2. <https://docs.z.ai/guides/llm/glm-5.2>. Developer documentation. Accessed: 2026-07-09.
 - [18] Ruizhe Zhong, Xingbo Du, Shixiong Kai, Zhentao Tang, Siyuan Xu, Hui-Ling Zhen, Jianye Hao, Qiang Xu, Mingxuan Yuan, and Junchi Yan. 2023. Llm4eda: Emerging progress in large language models for electronic design automation. *arXiv preprint arXiv:2401.12224* (2023).

A Final Product Scoring

Each entry is scored on a 0–100 scale:

$$S = 0.2S_{\text{stage}} + 0.8S_{\text{result}}.$$

A.1 Stage Score

The stage score consists of three sequential gates: logic synthesis, physical implementation (PR), and ECO.

The logic synthesis stage contains two checks: netlist generation and clock gating. Each check is worth 50 points. The PR stage contains four checks: IO placement, standard-cell placement, CTS, and routing. The ECO stage is a binary pass/fail gate.

The preliminary scores for the three stages are

$$G_L = 50(N_{\text{net}} + N_{\text{cg}}), \quad \tilde{G}_P = 25N_P, \quad \tilde{G}_E \in \{0, 100\}.$$

Here, N_{net} and N_{cg} each take a value in $\{0, 1\}$, indicating whether the netlist-generation check and the clock-gating check pass, respectively. The variable $N_P \in \{0, 1, 2, 3, 4\}$ denotes the number of passed PR checks.

The three gates are evaluated sequentially. A later stage contributes to the score only if all preceding stages have been fully passed. The PR gate score is defined as

$$G_P = \begin{cases} \tilde{G}_P, & G_L = 100, \\ 0, & G_L < 100. \end{cases}$$

Thus, the PR checks contribute to the score only when both logic synthesis checks have passed. Otherwise, the PR gate score is set to zero. The ECO gate score is defined as

$$G_E = \begin{cases} \tilde{G}_E, & G_L = 100 \text{ and } G_P = 100, \\ 0, & \text{otherwise.} \end{cases}$$

Thus, the ECO result contributes to the score only when both the logic synthesis stage and the PR stage have been fully passed. Otherwise, the ECO gate score is set to zero.

The overall process gate is

$$G_{\text{proc}} = \min(G_L, G_P, G_E),$$

and the stage score is

$$S_{\text{stage}} = \frac{G_L + G_P + G_E}{3}.$$

A.2 Result Score

The result score is enabled only when all three gates are fully passed:

$$S_{\text{result}} = \begin{cases} S_{\text{raw}}, & G_{\text{proc}} = 100, \\ 0, & \text{otherwise.} \end{cases}$$

Let

$$\mathcal{M} = \{A, W_s, T_s, W_h, T_h, P\},$$

where the six terms denote area, setup WNS, setup TNS, hold WNS, hold TNS, and power, respectively. The raw result score is

$$S_{\text{raw}} = \frac{1}{6} \sum_{m \in \mathcal{M}} S_m.$$

A.3 Metric Normalization

Area and power are smaller-is-better metrics. By default, they are normalized within the same frequency group. For a measured value x and a reference best value b , the normalized score is

$$S_{\text{small}}(x; b) = \begin{cases} 100, & 0 < x \leq b, \\ 100b/x, & x > b, \\ 0, & \text{otherwise.} \end{cases}$$

The fixed reference values are

Frequency	b_A (μm^2)	b_P (mW)
700 MHz	34291.88	6.92488
350 MHz	23746.27	2.49435

The total power is computed as

$$P = P_{\text{setup}} + P_{\text{hold}}.$$

Timing metrics are larger-is-better metrics. For a timing value x and a negative floor $L < 0$, the normalized score is

$$S_{\text{timing}}(x; L) = \begin{cases} 100, & x \geq 0, \\ 0, & x \leq L, \\ 100(x - L)/(-L), & L < x < 0. \end{cases}$$

The default timing floors are

$$L_{W_s} = -0.80, \quad L_{T_s} = -200.0, \quad L_{W_h} = -0.30, \quad L_{T_h} = -80.0.$$